

1 Jak funguje ListView + databáze

Princip je **jednoduchý a modulární**:

a) Model (Movie)

- Třída Movie obsahuje **data jednoho filmu**:

```
class Movie {  
    final int id; // primární klíč v databázi  
    final String title;  
    final String imageUrl;  
    final int rating;  
    final bool watched;  
}
```

- Model se používá jako **most mezi databází a UI**.
-

b) Service (MovieService)

- MovieService je **vrstva pro práci s databází** (Supabase).
 - Funkce:
 - `getMovies()` → vrátí seznam všech filmů z databáze
 - `addMovie(Movie movie)` → vloží nový film
 - `deleteMovie(int id)` → smaže film podle ID
 - Výhoda: **UI netuší, odkud data přichází** – může být lokální seznam nebo Supabase.
-

c) UI – HomePage

- HomePage má **FutureBuilder** nebo stavový seznam `List<Movie>`
- Když zavoláš `getMovies()`, data se načtou → `ListView` je zobrazí
- Každý film je položka `ListTile` nebo `Card` s obrázkem, názvem, ratingem a ikonou “watched”

DB (Supabase) <-- MovieService --> List<Movie> --> ListView --> user sees UI

- Pokud přidáš nový film přes formulář → zavoláš `addMovie()` → po refreshi se seznam aktualizuje → `ListView` zobrazí nový film.
-

2 Přidávání filmů

Existují 2 možnosti:

1. Ruční přidávání

- Vyplníš název filmu, rating, watched atd.
- Obrázek můžeš vložit jako URL (např. z Google nebo Wikipedia)

2. Automatické vyhledání obrázků

- Existuje několik veřejných API, např.:
 - **The Movie Database (TMDb)** – velmi populární, má cover images, posters, info o filmu
 - **OMDb API** – taky vrací URL posteru podle názvu filmu
- Princip:

1. Uživatel zadá název filmu
2. App zavolá API (např. TMDb) a získá URL plakátu
3. URL se uloží do Supabase + zobrazí v ListView

Příklad s TMDb (anglicky)

1. Zaregistruješ se na <https://www.themoviedb.org/> → získáš API key
2. Flutter request:

```
import 'dart:convert';
import 'package:http/http.dart' as http;

Future<String?> fetchPoster(String movieName) async {
  final apiKey = 'YOUR_TMDB_API_KEY';
  final url = Uri.parse(
    'https://api.themoviedb.org/3/search/movie?api_key=$apiKey&query=$movieName'
  );

  final response = await http.get(url);

  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);
    if (data['results'].length > 0) {
      return 'https://image.tmdb.org/t/p/w500' + data['results'][0]['poster_path'];
    }
  }
}
```

```
}  
}  
return null; // žádný obrázek nenalezen  
}
```

Pak můžeš použít:

```
final posterUrl = await fetchPoster('Interstellar');  
final movie = Movie(  
  title: 'Interstellar',  
  imageUrl: posterUrl ?? 'fallback_image_url',  
  rating: 5,  
  watched: true  
);  
await movieService.addMovie(movie);
```

3 Co se děje, když spustíš aplikaci

1. HomePage zavolá MovieService.getMovies()
2. getMovies() načte seznam filmů z databáze
3. Seznam se uloží do List<Movie>
4. ListView vytvoří widget pro každý film → zobrazí název, rating, poster, ikonku watched
5. Pokud přidáš nový film přes formulář → seznam se znovu načte / refresh → ListView se aktualizuje

Zatím můžeš začít s **lokálním seznamem**, jak jsme udělali, a až budeš mít Supabase, jen v MovieService nahradíš lokální seznam voláním do databáze.

4 Shrnutí

- **Model → Service → UI**
- ListView jen **zobrazuje data**, nevytváří je
- **Databáze** slouží pro **trvalé ukládání dat**
- **Poster obrázky** se dají:
 - Ručně vložit URL
 - Automaticky stáhnout z TMDb / OMDb API podle názvu filmu

Pokud chceš, můžu ti hned připravit **verzi formuláře na přidání filmu**, která:

- zadáš název filmu
- automaticky stáhne obrázek z TMDb
- přidá film do lokálního seznamu (zatím bez Supabase)

Tím už bude tvoje appka **interaktivní a pěkná pro prezentaci**.